Лабораторна робота №1. HTML і CSS для інтерфейсів

Мета

Створити доступну, адаптивну вебсторінку з семантичним HTML і CSS. Робота є підготовкою до компонування інтерфейсів у React і React Native: студенти вчаться бачити структуру, дані та стилі як окремі частини UI.

Результати навчання

Після виконання роботи студент уміє:

- створювати валідний HTML5-документ;
- застосовувати семантичні елементи (`header` , `main` , `section` , `footer`);
- пов'язувати HTML і CSS, користуватися класами та CSS-змінними;
- будувати mobile-first макет за допомогою Flexbox;
- додавати базову доступність: `lang` , `alt` , видимий фокус, логічну ієрархію заголовків;
- перевіряти сторінку у вузькому й широкому вікні браузера.

Як працювати з цією лабораторною

HTML і CSS не потребують Node.js, npm або встановлення бібліотек. Достатньо браузера та текстового редактора, наприклад VS Code.

1. Відкрийте папку лабораторної у VS Code через **File → Open Folder**.
2. У провіднику файлів зліва відкрийте потрібний `index.html` .
3. Для першого перегляду можна двічі клацнути файл у Finder/Explorer або перетягнути його у вікно браузера. Після кожної зміни збережіть файл (`Cmd+S` або `Ctrl+S`) і оновіть сторінку в браузері.
4. За бажанням встановіть у VS Code розширення Live Preview чи Live Server. Воно лише автоматично оновлює сторінку після збереження; для виконання завдань воно не обов'язкове.
5. Якщо щось відображається не так, відкрийте інструменти розробника браузера клавішею `F12` або через меню **Inspect / Перевірити**. Вкладка **Elements** показує HTML, а **Console** — повідомлення про помилки, наприклад неправильний шлях до CSS або зображення.

Як читати HTML і CSS-файли

HTML-файл — це дерево вкладених елементів. Відступи не змінюють роботу браузера, але показують, який елемент міститься всередині іншого. Закривальний тег має відповідати відкривальному:

```
<main>
  <section>
    <h2>Заголовок розділу</h2>
    <p>Текст розділу.</p>
  </section>
</main>
```

Атрибут записують усередині відкривального тега. Наприклад, у `<a href="#contacts">Контакти</a>` атрибут `href` пояснює браузеру, куди веде посилання; текст між тегами бачить користувач. У `<img src="logo.svg" alt="Логотип">` `src` задає шлях до файлу, а `alt` є текстовим описом зображення.

CSS читається як набір правил. У фрагменті нижче `button` — селектор, а кожен рядок усередині фігурних дужок — властивість і її значення:

```
button {
  background-color: #1455c0;
  color: white;
}
```

Крапка перед назвою означає клас (`.event-card`), решітка — ID (`#logo`). Клас можна використати для багатьох елементів, а ID — лише для одного унікального елемента на сторінці.

Короткі теоретичні відомості та підказки

HTML — структура і зміст

HTML (HyperText Markup Language) — це мова розмітки. Вона не «програмує» поведінку сторінки, а описує її вміст і будову: де заголовок, абзац, список, кнопка або зображення. Браузер читає HTML зверху вниз і будує з нього сторінку.

HTML-елемент найчастіше складається з відкривального тега, вмісту та закривального тега: `<p>Текст абзацу</p>`. Деякі елементи не мають текстового вмісту, наприклад `<img>`; важливу інформацію для нього передають атрибути: `<img src="logo.svg" alt="Логотип">`.

HTML описує **значення** вмісту, а не його вигляд. Браузер уже розуміє призначення елементів: `<button>` є кнопкою, `<time>` — датою/часом, `<main>` — головним вмістом. Тому обирайте елемент за його роллю, а не за тим, як його простіше стилізувати. Це робить код зрозумілішим, допомагає пошуковим системам і засобам читання з екрана.

- У кожному документі потрібні `<!doctype html>`, `<html lang="uk">`, `<head>` і `<body>`.
- `<!doctype html>` повідомляє браузеру, що це сучасний HTML5-документ. `<html lang="uk">` визначає основну мову, зокрема для читачів екрана.
- У `head` не розміщують видимий текст сторінки; тут розташовують `meta`, `title` та підключення CSS. У `body` міститься все, що побачить користувач.
- `meta charset="UTF-8"` забезпечує коректне відображення українського тексту, а `meta name="viewport" ...` — правильний масштаб на телефоні.

- `title` — назва вкладки браузера; вона має коротко описувати конкретну сторінку, наприклад `Campus Events — події`.
- На сторінці зазвичай один `h1`; заголовки `h2` – `h6` мають утворювати логічну ієрархію, а не вибиратися за розміром шрифту. Якщо назву потрібно зробити меншою, це роблять у CSS, а не заміною `h1` на `h3`.
- Використовуйте `header`, `main` і `footer` для великих частин сторінки; `section` — для тематичного розділу; `article` — для самостійної одиниці вмісту, наприклад картки події. Елемент `div` доречний лише тоді, коли контейнер не має власного змістового призначення.

CSS — правила оформлення

CSS (Cascading Style Sheets) відповідає за вигляд HTML: кольори, шрифти, відступи, розміщення елементів і реакцію макета на ширину екрана. CSS зазвичай підключають у `<head>` так: `<link rel="stylesheet" href="styles.css">`. Тоді зміни у файлі `styles.css` застосовуються до сторінки після її оновлення в браузері.

CSS-правило складається із селектора і набору властивостей:

```
.event-card {
  background-color: white;
  padding: 1rem;
}
```

Тут `.event-card` — селектор, а `background-color` і `padding` — властивості. Крапка означає клас: той самий клас можна додати кільком елементам. Селектор елемента (`h1`) оформлює всі такі елементи, клас (`.card`) — повторно використовуваний стиль, а ID (`#logo`) — один унікальний елемент. ID не слід повторювати на сторінці.

Коли кілька правил стосуються одного елемента, браузер застосовує каскад: більш конкретний селектор і правило, записане нижче за однаково конкретне, можуть перекрити попереднє. На початку навчання не варто розв’язувати такі конфлікти через `!important`: краще зробити селектори простими і передбачуваними.

CSS-змінні у `:root` зберігають повторювані значення, наприклад кольори: `--accent: #1455c0;`. Використання — `color: var(--accent);`. Якщо основний колір треба змінити, достатньо змінити його в одному місці. Це схоже на централізовані токени дизайну, які пізніше трапляться в мобільних застосунках.

Відступи й box model

Кожен елемент браузер розглядає як прямокутну «коробку». Її вміст оточують:

- `padding` — внутрішній відступ між вмістом і рамкою;
- `border` — рамка;
- `margin` — зовнішній відступ до інших елементів.

Правило `* { box-sizing: border-box; }` робить розміри передбачуванішими: оголошена `width` враховує і `padding`, і `border`. Для інтерфейсів це зручний базовий стандарт.

Адаптивність і Flexbox

Адаптивний дизайн означає, що одна сторінка зручно працює на телефоні, планшеті й комп'ютері. Починайте з вузького екрана (mobile-first): напишіть базові стилі для телефону, а ширший макет додавайте через `@media (min-width: 48rem)`. Умова в `@media` читається так: «застосуй ці додаткові стилі, коли ширина вікна щонайменше 48rem».

Не потрібно створювати окремий дизайн для кожної моделі телефону: гнучкі розміри, `max-width: 100%`, відносні одиниці та Flexbox дають змогу інтерфейсу пристосовуватися до доступної ширини. Наприклад, `max-width: 42rem; margin-inline: auto;` обмежує ширину картки на великому екрані та центрує її, але не ламає її на малому екрані.

`display: flex` розміщує дочірні елементи вздовж однієї осі — у рядок або стовпчик. Його задають батьківському елементу, а не кожній дочірній картці. Найуживаніші властивості:

- `flex-direction: column` — елементи розташовуються один під одним;
- `gap` — відстань між елементами;
- `justify-content` — вирівнювання вздовж головної осі;
- `align-items` — вирівнювання вздовж поперечної осі;
- `flex-wrap: wrap` — дозвіл переносити елементи на новий рядок, коли їм тісно.

Доступність — мінімум для цієї роботи

Доступність означає, що сторінкою можуть користуватися люди з різними способами взаємодії: клавіатурою, читачем екрана, збільшенням масштабу або за слабкого зору. Багато корисних речей дає сам HTML, якщо обрати правильний елемент.

- Змістовне зображення має короткий `alt`, який передає його функцію: `alt="Логотип ресторану Little Lemon"`. Декоративне зображення має порожній `alt=""`, щоб читач екрана його пропустив.
- Не перетворюйте `div` на кнопку стилями: використайте справжній `<button>`. Він уже працює з клавіатури через `Tab`, `Enter` і пробіл.
- Не прибирайте фокус клавіатури. Оформлюйте його через `:focus-visible`, щоб користувач бачив, на якому елементі перебуває.
- Колір не має бути єдиним способом передати важливу інформацію; текст і контраст залишаються читабельними. Наприклад, бейдж `NEW` містить слово, а не лише жовту позначку.
- Для дати/часу використовуйте `<time datetime="2026-09-15T16:00">15 вересня, 16:00</time>`: `datetime` зберігає машинозчитуваний формат, а текст усередині — зручний формат для людини.

Порядок виконання

1. Відкрийте папку лабораторної у VS Code.

2. Виконайте завдання послідовно: `task 1` → `task 2` → `task 3`.
3. Для перегляду відкрийте `index.html` у браузері або використайте розширення **Live Preview** / **Live Server**. Розширення не є обов'язковим.
4. Перед здачею пройдіть чеклісти, наведені після кожного завдання нижче.

Структура

- `task 1` — структура валідного HTML-документа;
- `task 2` — селектори, каскад, CSS-змінні та фокус;
- `task 3` — адаптивна картка події для мобільного застосунку.

Готовий демо-приклад

Папка `demo-приклад` містить завершену сторінку планувальника навчання. Вона використовує ті самі підходи, які потрібні в завданнях: семантичні елементи, CSS-змінні, Flexbox, адаптивність, видимий фокус клавіатури та `prefers-reduced-motion`.

Відкрийте спершу `demo-приклад/index.html` у браузері, а потім порівнюйте розмітку зі стилями у `demo-приклад/styles.css`. Не копіюйте приклад механічно: змініть його текст, кольори або розміщення елементів і перевірте, як це впливає на результат.

Завдання 1. Семантичний HTML-документ

Відкрийте `task 1/index.html`. У ньому вже є мінімальний технічний каркас — доповніть його так, щоб сторінка описувала майбутній мобільний застосунок **Campus Events**.

1. Перевірте наявність `<!doctype html>` і мови документа `lang="uk"`.
2. Перевірте кодування UTF-8 і viewport для мобільних пристроїв, а потім задайте змістовний `title`.
3. У `body` створіть `header`, `main` і `footer`.
4. У `header` розмістіть назву застосунку як `h1` та короткий опис.
5. У `main` додайте одну секцію з `h2`, абзацом і ненумерованим списком трьох подій.
6. У `footer` вкажіть поточний рік і нейтральний текст про навчальний макет.

Чекліст завдання 1

- На вкладці браузера є осмислена назва.
- Українські символи відображаються коректно.
- На сторінці є лише один `h1`.
- Структура містить `header`, `main` і `footer`, а не лише `div`.

Завдання 2. CSS: селектори, змінні та фокус

Відкрийте файли `task 2/index.html` і `task 2/styles.css`. У HTML уже є приклад меню Little Lemon. Допишіть стилі в CSS-файлі.

1. У `:root` створіть змінні для основного темного кольору, акцентного кольору, фону й відступу.
2. Застосуйте фоновий колір, системний шрифт, `line-height` і нульовий зовнішній відступ до `body`.
3. За селектором елемента оформіть `h1` та `h2`.
4. За селектором класу вирівняйте `.center-text` і обмежте його ширину.
5. За селектором ID зробіть логотип блоковим, адаптивним і відцентрованим. Не задавайте йому фіксовану ширину, більшу за екран.
6. За селектором нащадка підсвітіть `span` у `h2` як бейдж `NEW`.
7. Додайте стилі для посилання внизу сторінки та чіткий `:focus-visible` для клавіатурної навігації.
8. Додайте один media query від `48rem`, який збільшує горизонтальні відступи контейнера.

Чекліст завдання 2

- Сторінка не має горизонтального прокручування за ширини 320 px.
- Фокус на посиланні видно після натискання `Tab`.
- Кольори беруться з CSS-змінних, а не дублюються в багатьох правилах.

Завдання 3. Адаптивна картка події

Відкрийте файли `task 3/index.html` і `task 3/styles.css`. Створіть картку **«Відкрита лекція: Мобільний UX»** для університетського застосунку.

1. Доповніть `index.html`, не змінюючи базових метаданих.
2. Використайте `main`, `article`, `header`, `time` і `footer`.
3. Додайте назву, дату й час через елемент `time` з атрибутом `datetime`, місце проведення, короткий опис і список із трьох тез.
4. Додайте кнопку `Зареєструватися` з `type="button"`. Вона не має виконувати дію: JavaScript у цій роботі не потрібний.
5. У `styles.css` створіть mobile-first картку: світлий фон, читабельний текст, відступи, заокруглення та тінь.
6. Використайте Flexbox для розміщення метаданих. За ширини від `48rem` картка не має ставати ширшою за `42rem` і мусить лишатися по центру.
7. Оформіть `button:hover` і `button:focus-visible`; фокус не можна прибирати.
8. Додайте `@media (prefers-reduced-motion: reduce)`, який вимикає анімації/переходи.

Додатково

Додайте варіант картки з класом `event-card--featured`, який відрізняється лише акцентною рамкою. Не дублюйте всі стилі картки.

Чекліст завдання 3

- Макет читабельний на 320 px і 1024 px.
- Для назви є один `h1`, у картці — `h2`.
- Дата доступна не лише візуально, а й у `datetime`.
- Кнопка має видимий фокус після `Tab`.
- У макеті немає особистих фото, профілів або персональних контактів.

Здача

Надішліть папку з трьома виконаними завданнями або посилання на Git-репозиторій. Не додавайте `node_modules`, архіви браузера чи особисті дані.

Захист лабораторної роботи

Захист відбувається усно з демонстрацією виконаних завдань у браузері та редакторі коду. Студент має самостійно відкрити свою роботу, пояснити її будову й виконати одну коротку зміну, запропоновану викладачем.

Під час захисту потрібно:

1. Показати всі три завдання та відкрити їх у браузері.
2. Пояснити структуру HTML-документа: призначення `head`, `body`, `header`, `main`, `section` / `article` і `footer`.
3. Пояснити, як підключено CSS, які селектори використано та за що відповідають ключові стилі.
4. Продемонструвати адаптивність: змінити ширину вікна браузера й показати, як реагує макет.
5. Пояснити щонайменше дві доступнісні особливості роботи: наприклад, `alt`, семантичний `button`, `:focus-visible`, `lang` або `time`.
6. Внести мінімальну зміну в код без сторонньої допомоги: змінити текст чи дані події, колір через CSS-змінну, відступ, селектор, або додати один елемент списку. Після зміни потрібно оновити сторінку та пояснити результат.

Орієнтовні контрольні питання

- Чим відрізняються HTML і CSS?
- Для чого потрібні `<!doctype html>`, `lang`, `charset` і `viewport`?
- Коли слід використати `section`, `article` і `div`?
- Чому на сторінці бажано мати один `h1`?
- Чим відрізняються селектори елемента, класу та ID?
- У чому різниця між `margin`, `border` і `padding`?

- Що робить `display: flex` і чому його задають батьківському елементу?
- Що таке mobile-first і для чого застосовують `@media`?
- Для чого потрібні `alt` і `:focus-visible`?
- Чому для дії користувача краще використати `<button>`, а не стилізований `<div>`?

Корисні джерела

- MDN: базова структура HTML-документів
- MDN: семантичний HTML і доступність
- MDN: базові CSS-селектори
- MDN: Flexbox
- MDN: адаптивний вебдизайн і media queries
- MDN: доступні посилання, `alt` та інші HTML-практики

Важливо

- Не використовуйте справжні персональні дані, фото профілю або посилання на приватні соцмережі.
- Не потрібно підключати JavaScript, фреймворки або CSS-бібліотеки.
- Значення кольорів і текст у завданні можна змінювати, якщо зберігаються вимоги до структури й доступності.

Типові проблеми та їхнє вирішення

Проблема	Найімовірніша причина	Що перевірити
CSS не застосовується	Неправильний шлях у `href` або файл не збережено	`<link rel="stylesheet" href="styles.css">`, назву файла і оновлення сторінки
Зображення не видно	Неправильний `src` або інша назва/розширення файла	Шлях від `index.html`, великі/малі літери, наприклад `logo.svg`
Елементи «виїжджають» за екран	Фіксована ширина або великі відступи	`max-width: 100%`, `box-sizing: border-box`, ширину вікна 320 px
Стиль не змінюється	Селектор не відповідає HTML або інше правило його перекриває	Назву класу/ID, вкладеність елементів і нижчі правила у CSS
Клавіша `Tab` не показує фокус	Фокус прибрано або елемент не є інтерактивним	`:focus-visible` і використання `<button>` чи `<a href="...">`